

Feb 10, 2025 15:35

ControlDB.py

Page 1/14

```
1  # this controls the database interactions in pyside6
2  import sys
3  import os
4  import platform
5  from loguru import logger
6
7
8  import config_mycal
9  from Recipe1 import Ui_MainWindow
10
11
12  from PySide6.QtCore import (QSize, Qt ,QRect,
13  QCoreApplication,Slot,Signal,QAbstractListModel,QAbstractTableModel,QMetaObject,QModelIndex)
14  from PySide6.QtGui import QAction,QDoubleValidator,QFont
15  from PySide6.QtWidgets import QWidget
16  from PySide6.QtUiTools import QUiLoader
17
18  from PySide6.QtSql import QSqlDatabase , QSql,QSqlTableModel,QSqlQueryModel,QSqlQuery
19
20  from PySide6.QtWidgets import (
21  QApplication,
22  QCheckBox,
23  QComboBox,
24  QDateEdit,
25  QDateTimeEdit,
26  QDial,
27  QDoubleSpinBox,
28  QFileDialog,
29  QFontComboBox,
30  QFormLayout,
31  QHBoxLayout,
32  QLabel,
33 QLCDNumber,
34  QLineEdit,
35  QListView,
36  QMainWindow,
37  QProgressBar,
38  QPushButton,
39  QRadioButton,
40  QSlider,
41  QSpinBox,
42  QStatusBar,
43  QTimeEdit,
44  QTableView,
45  QMenuBar,
46  QVBoxLayout,
47  QAbstractItemView,
48  QGridLayout,
49  QTextEdit
50  )
51
52
53  class NewWindow(QMainWindow):
54      def __init__(self,Title = None):
55          super().__init__()
56          layout = QVBoxLayout()
57
```

Feb 10, 2025 15:35

ControlDB.py

Page 2/14

```

58         self.label = QLabel("Another Window")
59         if>Title == None):
60             self.setWindowTitle("MyCal")
61         else:
62             self.setWindowTitle>Title)
63
64         layout.addWidget(self.label)
65         self.setLayout(layout)
66
67
68     class RecipeModel(QAbstractTableModel):
69
70         def __init__(self, recipes=None, header_labels=None):
71             super().__init__()
72
73         #When subclassing QAbstractListModel, you must provide implementations of the rowCount() and data() functions.
74         # Well behaved models also provide a headerData() implementation.
75         # this means we will have to define the data and rowCount
76         # see also https://doc.qt.io/qt-6/qabstractlistmodel.html
77
78         self.recipes = recipes or [[]] #recipes will be a 2d array or a nested list
79         self.header_labels = header_labels = ["weight", "unit", "ingredient"]
80         def data(self, index, role):
81             if role == Qt.DisplayRole or Qt.ItemDataRole.DisplayRole or role == Qt.ItemDataRole.EditRole:
82                 value = self.recipes[index.row()][index.column()]
83                 # check for type
84                 if isinstance(value, float):
85
86                     return "%.2f" % value
87
88                 if isinstance(value, str):
89                     return "%s" % value
90
91             return value
92
93
94
95
96         def rowCount(self, parent=QModelIndex()):
97             return len(self.recipes)
98
99         def columnCount(self, parent=QModelIndex()):
100             return len(self.recipes[0])
101
102         def headerData(self, section, orientation, role=Qt.DisplayRole):
103             if role == Qt.DisplayRole and orientation == Qt.Horizontal:
104                 return self.header_labels[section]
105             return None
106
107         def insertRows(self, position, rows=1, parent=QModelIndex(), *args, **kwargs):
108             self.beginInsertRows(parent, position, position + rows - 1)
109             for _ in range(rows):
110                 self.recipes.insert(position, [""] * self.columnCount())
111             self.endInsertRows()
112             return True
113
114         def setData(self, index, value, role):

```

Feb 10, 2025 15:35

ControlDB.py

Page 3/14

```

115         if role == Qt.ItemDataRole.EditRole:
116             self.recipes[index.row()][index.column()] = value
117             return True
118         return False
119     def flags(self, index):
120         return Qt.ItemFlag.ItemIsSelectable | Qt.ItemFlag.ItemIsEnabled | Qt.ItemFlag.ItemIsEditable
121
122     def StoreValue(self):
123         return self.recipes
124
125
126 class MyRecipeWindow(QMainWindow, Ui_MainWindow):
127     def __init__(self):
128         super().__init__()
129         self.setupUi(self)
130
131
132 class ControlDB(QMainWindow):
133
134     def __init__(self, Title=None, db_name=None, db_user=None, db_system=None, db_pwd = None):
135         super().__init__()
136         self.db_name = db_name
137         self.db_user = db_user
138         self.db_system = db_system
139         if db_pwd == None:
140
141             temp = '/Users/klein/git/qt_exercises/config/pw.txt'
142             if os.path.exists(temp):
143                 with open(temp, 'r') as file:
144                     password = file.read().rstrip()
145                     self.db_pwd = password
146             else:
147                 self.db_pwd = db_pwd
148
149
150         self.SetupLogger()
151         #self.CreateIngredientsForm()
152         self.ingredients_suppress_columns = ['Sugar',
153                                             'Portion_size',
154                                             'Saturated',
155                                             'Fiber',
156                                             'Salt',
157                                             'Sodium',
158                                             'Product_source'
159                                             ]
160
161         self.recipe_suppress_columns = ['Sugar',
162                                         'Portions',
163                                         'Description',
164                                         'Saturated_fat',
165                                         'Fiber',
166                                         'Salt',
167                                         'Sodium',
168                                         'Ingredients',
169                                         'Time',
170                                         'Images',
171                                         'vegetarian' ]

```

Feb 10, 2025 15:35

ControlDB.py

Page 4/14

```

172
173     self.version = '0.3'
174     logger.info('*****\n\n\n')
175
176     logger.info(' This is version %s' % self.version)
177     logger.info('\n\n\n*****\n')
178
179
180
181
182     def ConnectDataBase(self):
183         ''' establish contact to database'''
184
185
186         #instantiate the connection
187         self.mycal_db = QSqlDatabase.addDatabase(self.db_system)
188         self.mycal_db.setHostName("localhost")
189         self.mycal_db.setDatabaseName(self.db_name)
190         self.mycal_db.setUserName(self.db_user)
191         self.mycal_db.setPassword(self.db_pwd)
192
193
194     #         self.mycal_db.setPassword(self.CM.pwd)
195
196     result = self.mycal_db.open()
197     if(result):
198         logger.info('connection successful')
199         # here we get the connection name
200         # will be needed when we want to close the connection
201         self.connection_name = self.mycal_db.connectionName()
202         logger.info(' database connection name %s' % self.connection_name)
203         #self.ShowTables()
204     else:
205         logger.error('connection failed, exiting')
206         sys.exit(0)
207
208     def SetupLogger(self):
209
210
211         logger.remove(0)
212         #now we add color to the terminal output
213         logger.add(sys.stdout,
214                   colorize = True, format="<green>{time}</green> {function} {line} {level} <level>{message}</level>" ,
215                   level = "INFO")
216
217
218
219         fmt = "{time} - {name} - {function} - {line} - {level} - {message}"
220         logger.add('info.log', format = fmt , level = 'INFO', rotation="1 day")
221
222
223         # set the colors of the different levels
224         logger.level("INFO", color = '<black>')
225         logger.level("WARNING", color = '<green>')
226         logger.level("ERROR", color = '<red>')
227         logger.level("DEBUG", color = '<blue>')
228

```

```

229         return
230
231
232
233     def ShowTables(self):
234         '''prints all the tables in the database'''
235         self.table_list = self.mycal_db.tables(type=QSql.Tables)
236         for k in range(0, len(self.table_list)):
237             print(self.table_list[k])
238
239         #Create list box
240         self.mybox = QComboBox()
241         #Insert at index 0 the whole list
242         self.mybox.insertItems(0, self.table_list)
243         # create independent window
244
245         #self.setCentralWidget(self.mybox)
246
247
248     return
249
250     def ViewTable(self, table, suppress_columns=[], editmode=True, columnwidth=[], Title = None, showTable = True):
251         self.table_view = QTableView()
252
253
254         self.model = QSqlTableModel(db = self.mycal_db)
255
256
257
258         self.table_view.setModel(self.model)
259         #prevent editing
260         if(not editmode):
261             self.table_view.setEditTriggers(QAbstractItemView.NoEditTriggers)
262
263         self.model.setTable(table)
264
265         # get rid of columns
266         columns_to_remove = suppress_columns
267         for cn in columns_to_remove:
268             idx = self.model.fieldIndex(cn)
269             self.model.removeColumns(idx, 1)
270
271
272         self.model.select()
273         # change column width
274         for k in range(0, len(columnwidth)):
275             self.table_view.setColumnWidth(k, columnwidth[k])
276
277         self.setMinimumSize(QSize(300, 300))
278
279         self.setCentralWidget(self.table_view)
280
281         self.SetPosition(10,10)
282         if(showTable):
283             self.show()
284         #self.setCentralWidget(table)
285

```

Feb 10, 2025 15:35

ControlDB.py

Page 6/14

```

286
287 def SetPosition(self, pos_x, pos_y):
288     self.move(pos_x, pos_y)
289
290 def CreateIngredientsForm1(self):
291     """uses the form layout"""
292
293     #always use self so window will be persistent
294     self.Inform = NewWindow(Title = "Add New Ingredient")
295     self.Inform.setStyleSheet("background-color: yellow;")
296
297     # setup geometry
298     mysize=[400,300]
299     myposit = [800,100]
300
301     self.SizeWindow(self.Inform, myposit, mysize)
302
303     # Create widgets
304     self.name_label = QLineEdit()
305     self.name_label.setStyleSheet("background-color: rgb(3, 252, 227)")
306     # force valid entr
307     double_validator = QDoubleValidator(0, 500.0, 2)
308
309     self.calory_label = QLineEdit()
310     self.calory_label.setStyleSheet("background-color: white")
311     self.calory_label.setValidator(double_validator)
312
313     self.carb_label = QLineEdit()
314     self.carb_label.setStyleSheet("background-color: white")
315     self.carb_label.setValidator(double_validator)
316     self.fat_label = QLineEdit()
317     self.fat_label.setStyleSheet("background-color: white")
318     self.fat_label.setValidator(double_validator)
319     self.prot_label = QLineEdit()
320     self.prot_label.setStyleSheet("background-color: white")
321     self.prot_label.setValidator(double_validator)
322
323     SaveButton = QPushButton("Save")
324     SaveButton.setStyleSheet("background-color: white")
325     CancelButton = QPushButton("Cancel")
326     CancelButton.setStyleSheet("background-color: red")
327
328
329     # Create layout
330     form_layout = QFormLayout()
331     form_layout.addRow("Name:", self.name_label)
332     form_layout.addRow("Calories/100g", self.calory_label)
333     form_layout.addRow("Fat", self.fat_label)
334     form_layout.addRow("Carbohydrates", self.carb_label)
335     form_layout.addRow("Protein", self.prot_label)
336     form_layout.addRow(SaveButton)
337     form_layout.addRow(CancelButton)
338
339     mylayout = QVBoxLayout()
340     mylayout.addLayout(form_layout)
341     self.setLayout(mylayout)
342

```

Feb 10, 2025 15:35

ControlDB.py

Page 7/14

```

343         # Connect submit button
344         SaveButton.clicked.connect(self.SaveIngredients)
345         CancelButton.clicked.connect(lambda : self.Cancel(self.Inform))
346
347         widget = QWidget()
348         widget.setLayout(myLayout)
349
350
351
352
353
354         self.Inform.setCentralWidget(widget)
355
356         #for label in MyLabel:
357         #     widget = QWidget()
358         #     widget.setLayout(layout)
359         #self.setCentralWidget(widget)
360
361
362         # create the form
363
364         self.Inform.show()
365
366
367     def InsertRecord(self, table = None):
368         """ this inserts a record into the cosen table
369         I will use the query function to do this.
370         The record contains the info to be added
371         record=[name,energy,fat,carbohydrate,protein]
372
373         This is the same for ingredients and receipes"""
374
375         #Create table view
376         if (table == "ingredients") :
377
378             self.ViewTable(table = table,suppress_columns=self.ingredients_suppress_columns,showTable= False)
379         else:
380             self.ViewTable(table = table,suppress_columns=self.recipe_suppress_columns)
381
382         self.model = QSqlQueryModel()
383         self.table_view.setModel(self.model)
384         #next we check we don't have an entry yet
385
386         #Form the sql INSERT statement
387         #     self.record = [Ing_name,Ing_calory,Ing_carb,Ing_fat,Ing_prot]
388         #INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', 9.99);
389         #INSERT INTO table (name,energy,carbohydrate,fat,protein) VALUES (self.record[0],
390         #                                                                    self.record[1],
391         #                                                                    self.record[2],
392         #                                                                    self.record[3],
393         #                                                                    self.record[4]);
394         temp = ''+self.record[0]+'''
395         sql = 'INSERT INTO '+table+' (name,energy,carbohydrate,fat,protein) VALUES ('+temp+','+str(self.record[1])+','+str(self.record[2])+','+str(
self.record[3])+','+str(self.record[4])+');'
396         print (sql)
397
398         query = QSqlQuery(sql,db=self.mycal_db)

```

```

399         self.model.setQuery(query)
400         logger.info(' inserted record %s into table %s' % (self.record[0],table))
401
402
403
404
405     return
406
407 def MyRecipes_new(self):
408     """using the QTdesigner"""
409     self.MRD = MyRecipeWindow()
410     self.MRD.show()
411
412     # Load the model
413     #self.header_labels = header_labels = ["weight", "unit","ingredient"]
414
415     self.RecipeModel = RecipeModel()
416     self.MRD.RecipeTableView.setModel(self.RecipeModel)
417
418
419     #label the columns
420     header = self.MRD.RecipeTableView.horizontalHeader()
421
422
423     #poulate the combo box
424     self.MRD.RecipeListCombo.setObjectName(u"RecipeListCombo")
425     self.MRD.RecipeListCombo.addItem(self.GetRecipeList())
426
427
428     #here we define the actions
429     self.MRD.RecipeListCombo.currentTextChanged.connect(self.getSelectedRecipe)
430
431     self.MRD.RecipeTableView.clicked.connect(self.on_item_click)
432
433     # save button
434     self.MRD.SaveRecipe.clicked.connect(self.save_recipe)
435
436     #cancel button
437     self.MRD.CancelButton.clicked.connect(lambda : self.Cancel(self.MRD))
438
439     #add row button
440     self.MRD.AddRowButton.clicked.connect(self.add_row)
441
442     #new recipe button
443     #self.MRD.NewRecipeButton.clicked.connect(self.create_new_recipe)
444
445     #if line has change
446     self.MRD.NewRecipeName.returnPressed.connect(self.create_new_recipe)
447     #self.MRD.NewRecipeName.textChanged.connect(self.create_new_recipe)
448
449 def create_new_recipe(self):
450     """creates new recipe"""
451     #first get name
452     self.new_recipe_name = self.MRD.NewRecipeName.text()
453     logger.info("new recipe name %s" % self.new_recipe_name)
454     # By default we add 10 new rows:
455     self.RecipeModel.insertRows(0,rows = 10)

```



```

456         return
457
458
459     def add_row(self):
460         """ adds row to reipe table"""
461         row_position = self.RecipeModel.rowCount()
462         self.RecipeModel.insertRows(row_position)
463
464         return
465
466     def on_item_click(self, index):
467
468         #columns and rows start from 0
469         item_text = index.data()
470         item_row = index.row()
471         item_column = index.column()
472         return
473
474
475         #print(f"Clicked item: {item_text}")
476         #print(index.row())
477
478     def save_recipe(self):
479         """gets called fro clicking the save button"""
480         record = self.RecipeModel.StoreValue()
481         logger.info("ingredients %s" % record)
482         #create original string
483         record = self.pack_record(record)
484         # Now update the record
485         self.update_record('recipes', 'ingredients', record, self.selected_recipe)
486
487         #remove window
488         self.MRD.close()
489
490
491         return
492
493     def update_record(self, table, column_name, record, row_name):
494         """ updates record for name in table
495         example SQL statement
496         UPDATE recipes SET ingredients = '30 g almonds 60 g dry_tomatoes 517 g tenderloin' WHERE name='basil_tenderloin';
497         """
498
499         sql = 'UPDATE ' + table + ' SET ' + column_name + ' = \'' + record + '\' WHERE name = \'' + row_name + '\';'
500         self.do_sql(sql)
501
502         return
503
504     def pack_record(self, record):
505         """ put record back into original form for ingredients, which is just one string"""
506         b = []
507         c = ''
508         a = record
509         for k in range(len(a)):
510             if (a[k][0] == ' '): break
511             b.append(a[k][0] + ' ' + a[k][1] + ' ' + a[k][2] + ' ')
512

```

Feb 10, 2025 15:35

ControlDB.py

Page 10/14

```

513
514     for k in range(0, len(b)):
515         c = c+ b[k]
516
517         # remove last space
518         d = c[0:len(c)-1]
519         return(d)
520
521
522     def getSelectedText(self, s):
523         self.selected_text = s
524         logger.info("You have selected %s text" % self.selected_text)
525         return
526
527
528     def getSelectedRecipe(self, s):
529         #Clear Buffer of recipes
530
531
532         self.RecipeModel.recipes = []
533
534         self.selected_recipe = s
535         logger.info("You have selected %s recipe" % self.selected_recipe)
536
537         # now we populate the listview
538         self.FillRecipeIngredientList()
539
540         # Now fill qlist
541         self.DisplayRecipeList()
542
543         # send signal list has been updated
544         self.RecipeModel.layoutChanged.emit()
545
546
547
548         return
549
550     def do_sql(self, sql):
551         """executes a sql statement"""
552         query = QSqlQuery(sql, db=self.mycal_db)
553         #model = QSqlQueryModel()
554         return
555
556
557     def FillRecipeIngredientList(self):
558         # here we get the ingredients from the Recipe database
559         # this is done with a query
560
561
562
563         sql = ' SELECT ingredients from recipes WHERE recipes.name LIKE \''+self.selected_recipe +'\';'
564         query = QSqlQuery(sql, db=self.mycal_db)
565         model = QSqlQueryModel()
566
567         model.setQuery(query)
568         while query.next():
569

```

Feb 10, 2025 15:35

ControlDB.py

Page 11/14

```

570         temp_value = ((query.value(0)))
571         # now we have to work with string
572         # the old database still had *u000a in front of the number
573         # need first to strip this
574         #check if temp contains *u000a
575         mask = 'u000a'
576         if(mask in temp_value):
577             temp_new = self.StripUnicode(temp_value,mask)
578             #temp_new = self.StripUnicode(temp_new,'*')
579
580
581
582
583         else:
584             #already the new entry way
585             temp_new1 = []
586             temp_new = temp_value.split(' ')
587             for k in range(0,len(temp_new),3):
588                 temp_new1.append([temp_new[k],temp_new[k+1],temp_new[k+2]])
589             self.recipe_ingredients = temp_new1
590             return
591
592
593         delimiter = "g"
594         result = temp_new.replace('u000a','')
595
596
597         result1 = self.split_and_keep(result, delimiter)
598         # strip last *
599         result11= result1[0]
600         result2 = result11[:len(result1[0])-1].split('*')
601         # we know that a string with underscores has a max of three spaces. Any number larger than this means words with spaces.
602         #in other words 4 spaces means the 3rd has to be replaced with _ 5 spaces mean the 3rd and 4th
603         result3=[]
604         for a in result2:
605             space_indices = []
606             for i, char in enumerate(a):
607                 if char == ' ':
608                     space_indices.append(i)
609
610             if len(space_indices) == 4:
611                 na = a[:space_indices[2]] + '_' + a[space_indices[2] + 1:]
612
613                 result3.append(na)
614
615             elif len(space_indices) == 5:
616                 na = a[:space_indices[2]] + '_' + a[space_indices[2]+1:space_indices[3]] + '_' + a[space_indices[3] + 1:]
617                 result3.append(na)
618
619             else:
620                 result3.append(a)
621
622
623         recipe_string = result3
624         #finally split every field up into 3 '10 g bean' -> [10,'g','bean']
625         result4=[]
626         for i in range(len(result3)):

```

Feb 10, 2025 15:35

ControlDB.py

Page 12/14

```

627         temp = (result3[i].split(' '))
628         result4.append(temp[0:3])
629         self.recipe_ingredients = result4
630         return
631
632     def DisplayRecipeList(self):
633         """loop through ingredient list and add"""
634         for k in range(len(self.recipe_ingredients)):
635             self.RecipeModel.recipes.append(self.recipe_ingredients[k])
636         return
637
638
639     def split_and_keep(self, text, delimiter):
640         """deals with spaces instead of underscores"""
641         result = []
642         temp = ""
643         for char in text:
644             if char == delimiter:
645                 result.append(temp + char)
646                 temp = ""
647             else:
648                 temp += char
649         if temp:
650             result.append(temp)
651         return result
652
653
654
655
656     def StripUnicode(self, mystring, mask):
657         newstring = mystring.replace(mask, '')
658         return newstring
659
660
661     def GetRecipeList(self):
662         """sql query to get all the names of the recipies
663         will be used to populate the recipe qlistbox"""
664         sql = 'SELECT name from recipes;'
665         query = QSqlQuery(sql, db=self.mycal_db)
666         model = QSqlQueryModel()
667
668         model.setQuery(query)
669         recipe_list = []
670         while query.next():
671             recipe_list.append(str(query.value(0)))
672
673         return (recipe_list)
674
675
676
677
678
679     def ModifyRecipe(self):
680         pass
681
682
683     def SizeWindow(self, window, myposit, mysize):

```

```

684         """resizes the specified window ,where the parameters are two integer lists"""
685         window.setGeometry(myposit[0],myposit[1],mysize[0],mysize[1])
686         return
687
688
689 #####
690 # section for signals and slots
691
692
693 def Cancel(self,window):
694     """ closes window without anything"""
695     window.close()
696
697
698
699
700 def SaveIngredients(self):
701     """saving the ingredients"""
702     Ing_name      = self.name_label.text()
703     Ing_calory    = float(self.calory_label.text())
704     Ing_carb      = float(self.carb_label.text())
705     Ing_fat       = float(self.fat_label.text())
706     Ing_prot      = float(self.prot_label.text())
707
708     # this is the record we got from the form input
709
710     #Check all fields are filled out
711     if(Ing_calory !="" and Ing_carb !="" and Ing_fat!="" and Ing_prot != ""):
712         self.Inform.destroy() # destrorys the window.
713         #self.Inform.close() #
714
715     #now we need to add the values to the ingredients table
716     # first load table
717     table_name = 'ingredients'
718     self.ViewTable(table_name,suppress_columns=self.ingredients_suppress_columns)
719     self.model = QSqlQueryModel()
720     self.table_view.setModel(self.model)
721     #next we check we don't have an entry yet
722
723
724     #sql = "SELECT name from ingredients WHERE ingredients.name LIKE 'Almond%' ; "
725     sql = "SELECT name from ingredients WHERE ingredients.name LIKE '"+Ing_name+"' ; "
726     query = QSqlQuery(sql,db=self.mycal_db)
727
728     self.model.setQuery(query)
729     while query.next():
730         logger.error(' entry name already exists, try again %s' % query.value(0))
731         self.CreateIngredientsForm1()
732         #myvalue = query.value(0)
733     # Now we need to add this ingredient to the table
734
735     self.record = [Ing_name,Ing_calory,Ing_carb,Ing_fat,Ing_prot]
736     self.InsertRecord(table = table_name)
737     return
738
739
740 app = QApplication(sys.argv)

```

Feb 10, 2025 15:35

ControlDB.py

Page 14/14

```
741 Title=None
742 db_name='recipe_ak'
743 db_user='klein'
744 db_system='QPSQL'
745 db_pwd = None
746
747
748 suppress_columns = ['Sugar',
749                     'Portions',
750                     'Description',
751                     'Saturated_fat',
752                     'Fiber',
753                     'Salt',
754                     'Sodium',
755                     'Ingredients',
756                     'Time',
757                     'Images',
758                     'vegetarian']
759 columnwidth = [40,200,200,200,200,200]
760
761 window = ContrlDB(Title = "ControlDB",
762                  db_name=db_name,
763                  db_user=db_user,
764                  db_system=db_system)
765 #window.SetSize(800,500)
766 #window.SetPosition(100,500)
767
768 window.setStyleSheet("background-color: white;")
769 #window.CreateCalendar()
770
771
772
773 #window.show()
774 window.ConnectDataBase()
775 window.ShowTables()
776 #window.ViewTable('Recipes',suppress_columns=suppress_columns,columnwidth=columnwidth)
777 window.MyRecipes_new()
778 #window.CreateIngredientsForm1()
779
780 # now run the app
781 app.exec()
```